

Ch 03: Page Tables

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

- 1 Paging Hardware
- 2 Kernel Address Space
- 3 Process Address Space



Ch 03: Paging Hardware

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Page tables are the most popular mechanism through which an operating system provides each process with its own private address space and memory

- Page tables:
 - determine what memory addresses mean
 - determine what parts of physical memory can be accessed
 - allow **isolation** of different process' address spaces
 - allow to **multiplex** processes into a single physical memory
 - allow illusion that each process has full control of memory address space with no other processes able to access
- Page tables provide a level of indirection allowing some tricks used by xv6
 - map same memory (trampoline page) in several address spaces
 - guard kernel and user stacks with unmapped page to detect overflow
 - allocate user heap memory lazily



Paging Hardware (1 of 4)

- RISC-V instructions (both user and kernel) manipulate virtual addresses
- The machine's RAM, or physical memory, is indexed with physical addresses
- The RISC-V page table hardware maps each virtual address to physical address
- Xv6 uses RISC-V Sv39 mode
 - only bottom 39 bits (of 64) are used
 - RISC-V page table is array of 2^{27} (134, 217, 728) **page table entries** PTEs
 - Each PTE contains 44-bit physical page number PPN and flags
 - Paging hardware translates top 27 bits of virtual address into page table index to find PTE
 - Results in 56-bit physical address, 44-bits from PPN, and 12-bit offset

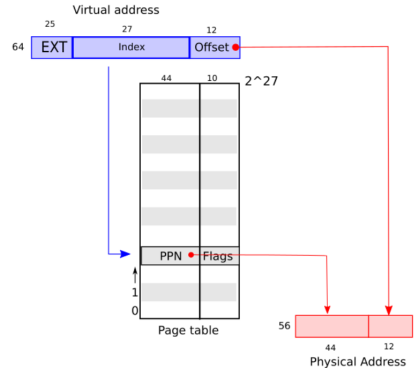


Figure 1: An abstract view of a flat page table mapping virtual to physical addresses (Cox et al., 2026, pg.32)



Paging Hardware (2 of 4)

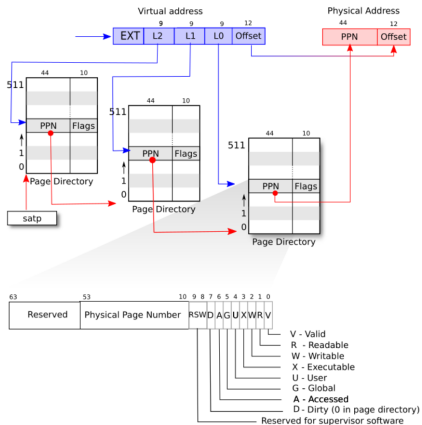


Figure 2: RISC-V address translation details (Cox et al., 2026, pg.33)

- A page table gives the operating system control over virtual-to-physical address translations
 - page size $2^{12} = 4096$ bytes `kernel/riscv.h PGSIZE`
 - Such a chunk is called a **page**
- Actual page table layout is a bit more complex, three-level tree
 - root of tree is 4096-byte page table containing 512 PTEs (8 bytes or 64 bits per PTE)
 - first 9 bits of virtual address index top level page table, next mid level, etc.
 - Allows memory-efficient way of managing PTEs
 - Potentially requires 3 memory accesses for each VA → PA translation
 - RISC-V CPU uses **Translation Look-aside Buffer (TLB)**



Paging Hardware (3 of 4)

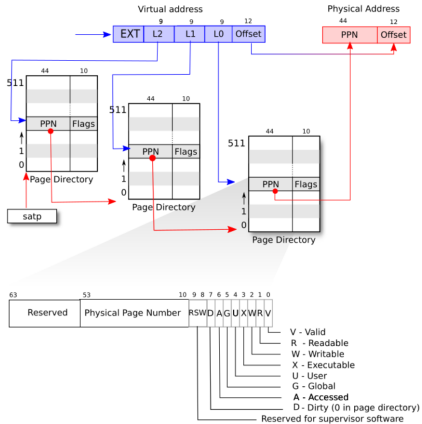


Figure 3: RISC-V address translation details (Cox et al., 2026, pg.33)

- PTE_V Valid, indicates whether the PTE is present, if not reference causes page fault
- PTE_R Readable, controls whether instructions are allowed to read the page
- PTE_W Writable, controls whether instructions are allowed to write to the page
- PTE_X Executable, controls whether CPU may interpret content of page as instructions and execute them
- PTE_U User, controls whether instructions in user mode are allowed to access page
- PTE_A Accessed, indicates if page has ever been accessed (read from) since created
- PTE_D Dirty, indicates if page has been written to



Paging Hardware (4 of 4)

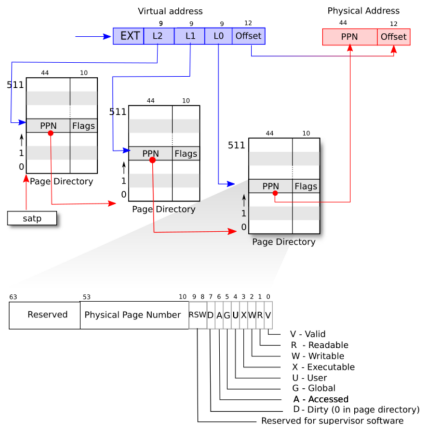


Figure 4: RISC-V address translation details (Cox et al., 2026, pg.33)

- To tell CPU to use page table, kernel writes physical address of root page-table into satp register
 - kernel/start.c disables paging initially
 - kernel/main.c calls `kvminithart()` in kernel/vm.c which enables kernel page table
 - Later each process has a page table created, and satp is set each time xv6 switches to user mode
- CPU translates all addresses by subsequent instructions using page table pointed to by satp
 - Each multicore CPU has own satp so each can run different processes with private address space
- From the kernel's point of view, page table is data stored in memory
 - kernel creates and modifies page tables using code much like any you would see for tree-shaped data structure



Summary

- Page tables provide processes with their own private address space
 - This is basic mechanism by which OS implements isolation and multiplexing of resources, like cpu time sharing
- The CPU architecture MUST support runtime translation from a virtual address to a physical address at runtime, using a defined page table structure
 - RISC-V uses satp register in supervisor and user mode as base of page table for translation
 - The Sv39 standard defines a 3-level page table tree
 - allows for 39 bit virtual addresses (38 used by xv6)
 - Requires 4 memory accesses to translate each memory reference to a physical address
- xv6 and RISC-V Sv39 use 12 bit page offsets, meaning page PGSIZE is $2^{12} = 4096$ bytes
- Each page directory is held on a single page and contains 512 page table entries



Ch 03: Kernel Address Space

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Kernel Address Space (1 of 2)

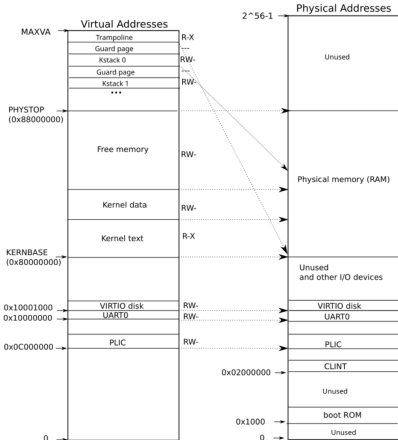
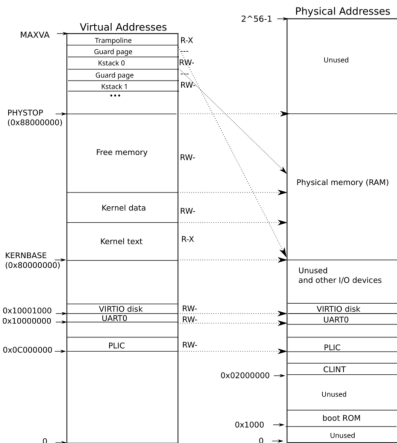


Figure 5: On the left xv6's kernel virtual address space. RWX refer to PTE read, write and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.

- When it starts, xv6 create a single page table describing the kernel's address space
 - configures layout of its address space to give it access to all of physical memory
 - configured to allow access to various hardware resources at predictable virtual addresses
 - memory-mapped** control registers that sit below 0x80000000
 - kernel maps all physical RAM at virtual addresses equal to the physical address
 - direct mapping**
 - configured in `kernel/memlayout.h`



Kernel Address Space (2 of 2)



- Most of the xv6 code for manipulating address spaces and page tables resides in `kernel/vm.c`

- `pagetable_t` is really a pointer to a root page-table page
- functions that start with `kvm` manipulate the kernel page table
- functions that start with `uvm` manipulate a user page table

Figure 6: On the left xv6's kernel virtual address space. RWX refer to PTE read, write and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.



Summary

- There is a page table that is created during boot that defines the kernel address space
- All of physical memory (including kernel code and data) is direct mapped from the kernel address space to a physical address
- Hardware devices are memory-mapped to addresses below KERNBASE and are accessible in kernel mode by device drivers
- Not all kernel addresses are direct mapped
 - The trampoline page is mapped to the top of the virtual address space, it actually is mapped 2 times in this space
 - Kernel stack pages for processes are also mapped to top of virtual address space, surrounded by guard pages to help prevent kernel bugs
- Functions starting with `kvm` in `kernel/vm.c` manipulate and create the kernel page table
 - These function are used before page table mapping is actually enabled during boot



Ch 03: Process Address Space

xv6 a simple Unix-like teaching Operating System

Derek Harter

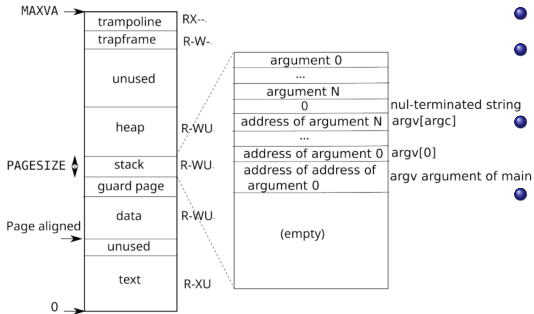
Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Process Address Space (1 of 2)

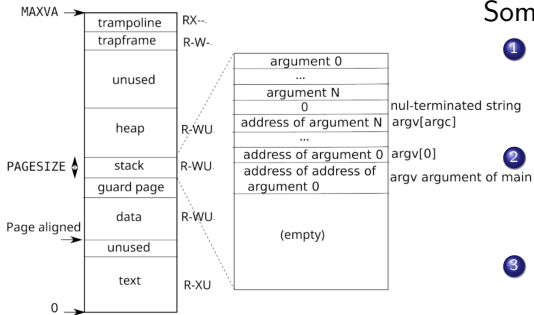


- Each process has its own page table
- When xv6 switches between processes, it also changes page tables
- User addresses start at 0 and end at MAXVA (0x4000000000) `kernel/riscv.h` near bottom
- A processes address space has pages with
 - text of the program (instructions, permissions are **PTE_R**, **PTE_X** and **PTE_U**)
 - pre-initialized data (constants, global variables **PTE_R**, **PTE_W**, **PTE_U**)
 - a page for the stack (**PTE_R**, **PTE_W**, **PTE_U**)
 - pages for dynamic heap (**PTE_R**, **PTE_W**, **PTE_U**)

Figure 7: A process's user address space, with its initial stack (Cox et al., 2026, pg.39)



Process Address Space (2 of 2)



Some properties of xv6 use of page tables here:

- 1 Different process' page tables translate user addresses to different pages of physical memory, so that each process has private user memory
- 2 Each process sees its memory as having contiguous virtual addresses starting at 0, while the process's physical memory can be non-contiguous
- 3 The kernel maps a page with trampoline code at the top of the user address space (without PTE_U), thus a single page of physical memory shows up in all address spaces, but can be used only by the kernel.

Figure 8: A process's user address space, with its initial stack (Cox et al., 2026, pg.39)



Summary

- Each process has its own page table
- The `exec()` system call (`kexec()` in `kernel/exec.c`)
 - Creates a new page table
 - Loads a binary file in elf format, initially the elf header which indicates number of segments in file
 - Then loads each text (instruction) and data segment into the new process virtual address space
 - Text (instruction) segments are loaded R-XU
 - Data segments are loaded R-WU
 - A stack is then created with a guard page, so that if it grows past end of stack a trap occurs
 - The initial `argv` argument values passed into `exec()` are copied onto initial stack
 - If successful, the pagetable, program counter and stack pointer are set in the trap frame, so that on return to user mode for this process, will start executing the newly loaded code at the entry point



Cox, R., Kaashoek, F., & Morris, R. (2026). *Xv6: A simple, unix-like teaching operating system*. Retrieved September 2, 2025, from <https://github.com/mit-pdos/xv6-riscv-book>

